

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 28 febbraio 2013 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

AVVISO - A PARTIRE DALLA SESSIONE ESTIVA (GIUGNO 2013), GLI ESERCIZI DI ANALISI SINTATTICA ANDRANNO AFFRONTATI SECONDO LA TEORIA *ELR* / *ELL* - LA TEORIA *LR* / *LL* NON SARÀ PIÙ AMMESSA - SI VEDA IL SITO WEB DEL CORSO PER MATERIALE DIDATTICO E INFORMAZIONI

NOME:

MATRICOLA:

FIRMA:

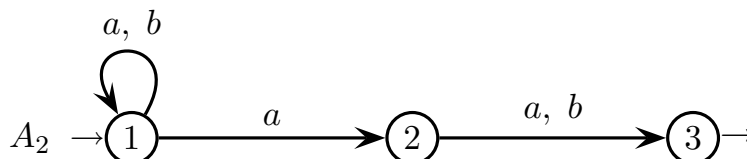
ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. Sono dati l'espressione regolare R e l'automata a stati finiti A_2 seguenti:

$$R = b^* (a b^+)^*$$



Si risponda alle domande seguenti:

- (a) Si consideri l'espressione regolare R data sopra:
- Si scrivano in ordine alfabetico le stringhe di lunghezza quattro appartenenti al linguaggio regolare $L(R)$ e si descriva a parole tale linguaggio.
 - Utilizzando il metodo Berry-Sethi, si ricavi un automa a stati finiti deterministico A_1 che riconosce il linguaggio regolare $L(R)$; se è necessario, si minimizzi l'automata A_1 .
 - (facoltativa) Si dica se il linguaggio regolare $L(R)$ è locale.
- (b) Si consideri l'automata a stati finiti A_2 dato sopra:
- Si dica quali delle stringhe date in risposta al punto (a).i sono accettate anche dall'automata A_2 .
 - Si costruisca l'automata prodotto $A_3 = A_1 \times A_2$ che accetta il linguaggio $L(R) \cap L(A_2)$, e si dica se l'automata prodotto A_3 è deterministico o no.
 - Si mostrino le tracce delle esecuzioni dell'automata prodotto che accettano le stringhe di cui al punto (b).i.
-

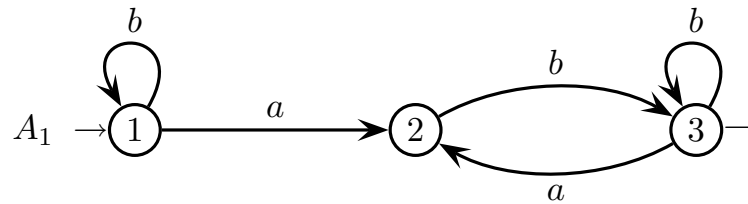
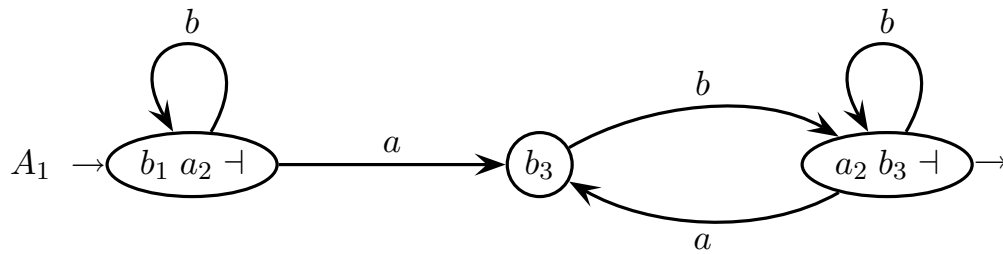
Soluzione

(a) Per la prima domanda:

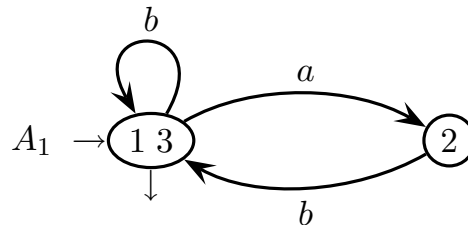
- i. Le stringhe sono cinque: $abab$, $abbb$, $babb$, $bbab$ e $bbbb$. Il linguaggio ha tutte e sole le stringhe dove ogni lettera a è seguita da una lettera b .
- ii. Ecco l'automa deterministico A_1 tramite il metodo di Berry-Sethi (BS).

$$R = b_1^* (a_2 b_3^+)^* \dashv$$

inizi	$b_1 a_2 \dashv$
gen.	seguiti
b_1	$b_1 a_2 \dashv$
a_2	b_3
b_3	$a_2 b_3 \dashv$

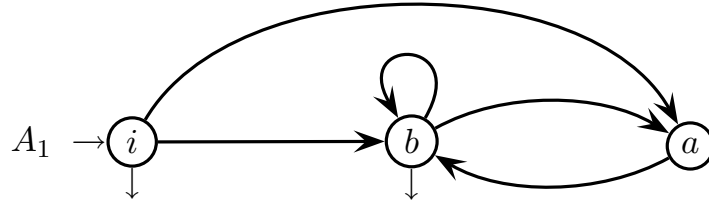


Si vede subito che gli stati 1 e 3 sono indistinguibili: sono finali, con ingresso a vanno nello stato 2, e con ingresso b restano in 1 e 3. Invece lo stato 2 è distinguibile sia da 1 sia da 3 poiché non è finale. Pertanto si possono unificare gli stati 1 e 3, così:



Ora l'automa deterministico A_1 è minimo.

- iii. Il linguaggio regolare $L(R)$ è locale. Infatti l'automa A_1 minimo che lo riconosce, ha due stati e in ciascuno di essi si entra con archi etichettati con la stessa lettera d'ingresso: a per 2 e b per 1 3. È immediato ridisegnare l'automa A_1 in forma locale, dove ogni stato è etichettato univocamente con una lettera d'ingresso, oltre allo stato iniziale. Eccolo:



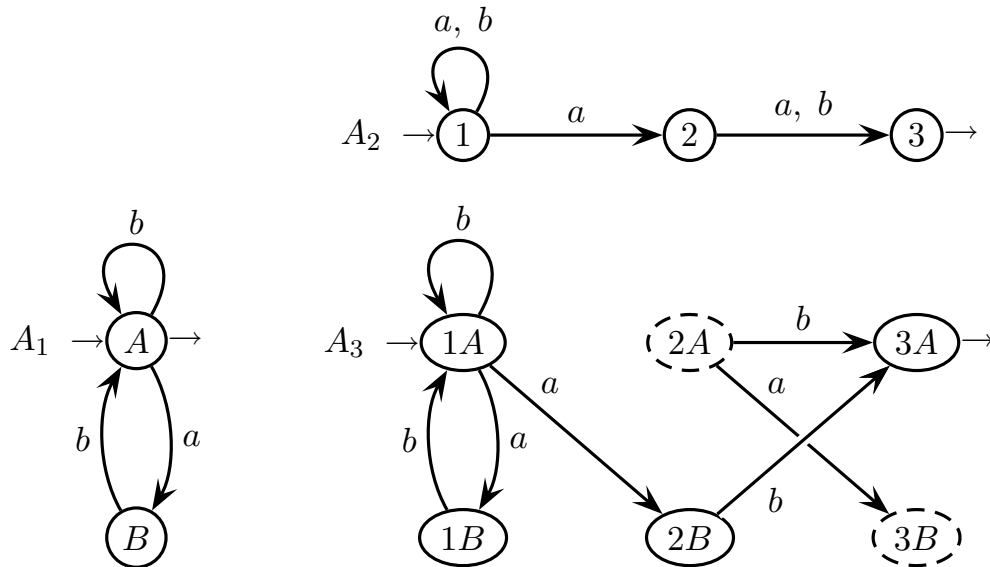
In termini di lettere iniziali, finali e digrammi, il linguaggio locale $L(R)$ è evidentemente descrivibile così:

$$Ini = \{ a, b \} \quad Fin = \{ b \} \quad Dig = \{ ab, ba, bb \}$$

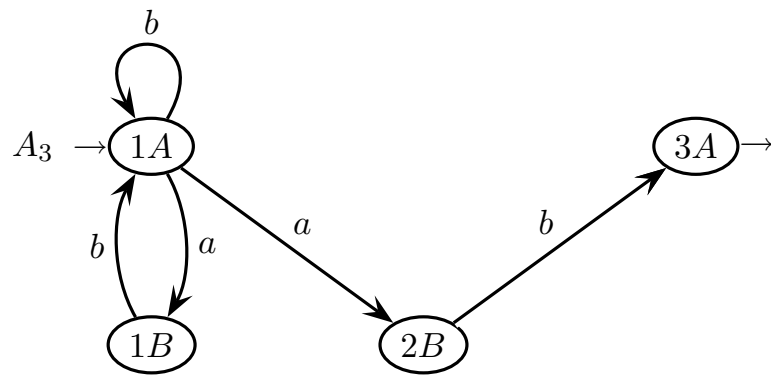
ovvero le stringhe di $L(R)$ non terminano con la lettera a e non contengono il digramma aa . A queste va aggiunta la stringa vuota ε .

(b) Per la seconda domanda:

- i. Le stringhe sono due: $abab$ e $bbab$. L'automa A_2 accetta le stringhe di lunghezza ≥ 2 dove il penultimo simbolo è la lettera a .
- ii. Ecco l'automa prodotto $A_3 = A_1 \times A_2$:



L'automa A_3 non è in forma ridotta poiché gli stati $2A$ e $3B$ sono irraggiungibili dallo stato iniziale, e lo stato $3B$ è indefinito e non raggiunge lo stato finale (sono in tratteggio), ma si riduce facilmente così:



L'automa prodotto A_3 non è deterministico e del resto non lo è neppure l'automa fattore A_2 . Naturalmente lo si potrebbe rendere deterministico.

iii. Ecco le computazioni richieste dell'automa prodotto A_3 :

$$\rightarrow 1A \xrightarrow{a} 1B \xrightarrow{b} 1A \xrightarrow{a} 2B \xrightarrow{b} 3A \rightarrow$$

$$\rightarrow 1A \xrightarrow{b} 1A \xrightarrow{b} 1A \xrightarrow{a} 2B \xrightarrow{b} 3A \rightarrow$$

Entrambe accettano la stringa rispettiva, come devono.

2 Grammatiche libere e automi a pila 20%

1. Preso l'alfabeto $\Sigma = \{0, 1, '(', ')'\}$, un *numero binario* $y \in \{0, 1\}^+$ è una stringa di bit. Il linguaggio $L_1 \subseteq \Sigma^*$ è definito ricorsivamente. Si supponga che le stringhe x , x_1 e x_2 siano frasi arbitrarie di L_1 . Valgono le tre condizioni ricorsive seguenti:

- ogni binario y appartiene a L_1
- la stringa “(x)” appartiene a L_1
- il concatenamento “ $x_1 x_2$ ” appartiene a L_1

Esempi e controesempi:

<i>valida</i>	00110	
<i>valida</i>	(0 (10) (110) 100 (10))	
<i>valida</i>	(1100) 0 ((10)) (11 (110) 0) 100	
<i>non valida</i>	(0 () (110) 100 (10))	- coppia vuota
<i>non valida</i>	(0 (11) (110)	- sbilanciamento

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G_1 , non estesa e non ambigua, per il linguaggio L_1 , e se ne giustifichi la non ambiguità.
- (b) In una frase di L_1 si consideri una coppia di parentesi corrispondenti. Il *numero binario* associato ad esse, è la sequenza di bit contenuti tra dette parentesi, ma non contenuti in parentesi più interne. Anche i bit eventuali che non sono racchiusi tra nessuna parentesi, costituiscono un numero binario. Si vedano gli esempi nella tabella successiva.

Si scriva una grammatica G_2 , non estesa, per il linguaggio $L_2 \subset L_1$ definito così:

$$x \in L_2 \text{ se e solo se } \begin{cases} x \in L_1 & \text{and} \\ \text{ogni numero binario in } x \text{ termina con il bit 0} \end{cases}$$

Esempi, dove i numeri binari cancellati sono quelli che invalidano la stringa:

		<i>numeri binari contenuti</i>
<i>valida</i>	0110 (10)	0110, 10
<i>valida</i>	0 (10) 100 (0 (0) 0)	0100, 10, 00, 0
<i>non valida</i>	(0 (11) (0) (10) 1)	01 , 11 , 0, 10
<i>non valida</i>	11	11

Soluzione

Language L_1 looks very similar to the Dyck language. One should remind that the Dyck language over the alphabet $\{ '(', ') ' \}$ can be defined by means of a simple *EBNF* rule: $S \rightarrow S^* \mid ' (' S ') '$; or by means of two rules like $S \rightarrow N^*$ and $N \rightarrow ' (' S ') '$, with axiom S , so as to single out the role of the corresponding parentheses N . A quick transformation into *BNF* is: $S \rightarrow N S \mid \varepsilon$ and $N \rightarrow (S)$.

To apply these known facts to the present case, one just needs to put in binary numbers, exclude void parentheses and recode all into *BNF*. This is done below.

(a) First here is a straightforward *EBNF* grammar for language L_1 (axiom S):

$$\left\{ \begin{array}{ll} S \rightarrow (N \mid B)^+ & \text{- non-empty sequence of nests } N \text{ and bits } B \\ N \rightarrow ' (' S ') ' & \text{- parenthesis nest} \\ B \rightarrow 0 \mid 1 & \text{- bit} \end{array} \right.$$

This *EBNF* grammar basically generates the Dyck language, where any two corresponding parentheses, i.e., a nest, cannot be void and may either contain a binary number, i.e., a sequence of one or more bits possibly interleaved with parenthesis nests, or just contain parenthesis nests not interleaved with any bits. Notice one might explicitly write the axiomatic rule as follows:

$$S \rightarrow (N \mid B)^* B N^* \mid N^+$$

where the alternative (between at least one bit or none) is evident.

Then it is mechanically converted into a *BNF* grammar G_1 by transforming the iterations into right recursions (axiom S):

$$G_1 \left\{ \begin{array}{ll} S \rightarrow N S \mid B S \mid N \mid B & \text{- } S \text{ equivalent to } (N \mid B)^+ \\ N \rightarrow (S) \\ B \rightarrow 0 \mid 1 \end{array} \right.$$

Grammar G_1 is basically a well-known Dyck formulation that notoriously has the *ELR*(1) property; so it is deterministic and therefore unambiguous as well. One can compact grammar G_1 into only one *BNF* rule, as follows (axiom S):

$$G'_1: S \rightarrow (S) S \mid 0 S \mid 1 S \mid (S) \mid 0 \mid 1$$

Of course, in such a form the grammar is less readable. It would be fairly more readable by going back to the extended form (axiom S):

$$G''_1: S \rightarrow (' (' S ') ' \mid 0 \mid 1)^+$$

or equivalently

$$G'''_1: S \rightarrow S^+ \mid ' (' S ') ' \mid 0 \mid 1$$

though the extended form is not required by the exercise.

- (b) Language L_2 constrains the phrases of language L_1 by requiring every binary number to end by zero. First the *EBNF* grammar is modified so as to comply with this constraint (axiom S), see also the observation before:

$$\left\{ \begin{array}{l} S \rightarrow (N \mid B)^* 0 N^* \mid N^+ \quad \text{- last bit is zero or no bits at all} \\ N \rightarrow '(S)' \\ B \rightarrow 0 \mid 1 \end{array} \right.$$

This *EBNF* grammar forces the last bit of every binary number, to be zero. Then a *BNF* grammar G_2 for language L_2 is mechanically obtained (axiom S):

$$G_2 \left\{ \begin{array}{l} S \rightarrow X 0 Y \mid Z \\ X \rightarrow N X \mid B X \mid \varepsilon \quad \text{- } X \text{ equivalent to } (N \mid B)^* \\ Y \rightarrow N Y \mid \varepsilon \quad \text{- } Y \text{ equivalent to } N^* \\ Z \rightarrow N Z \mid N \quad \text{- } Z \text{ equivalent to } N^+ \\ N \rightarrow (S) \\ B \rightarrow 0 \mid 1 \end{array} \right.$$

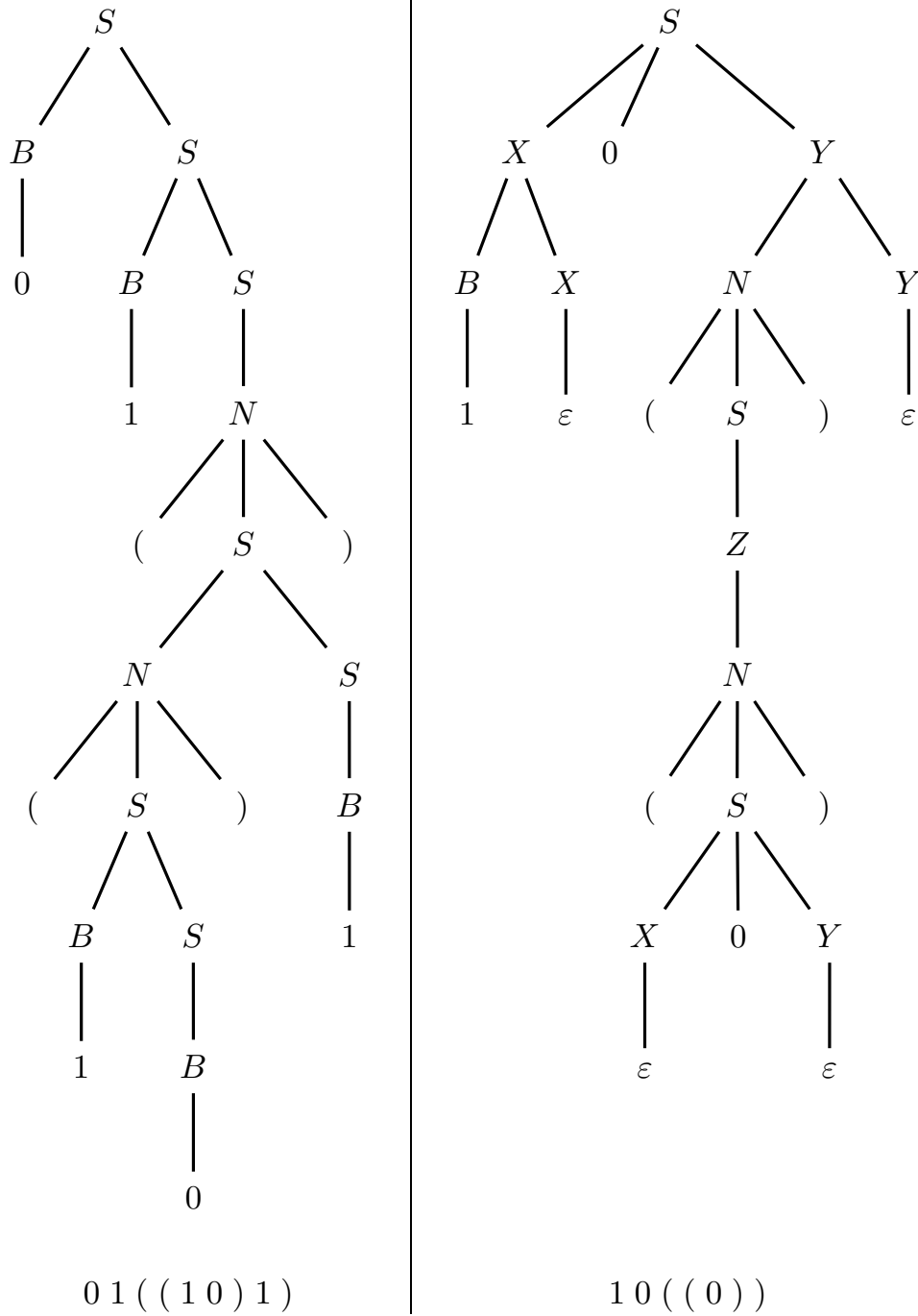
Grammar G_2 is also unambiguous, basically for the same reasons as before (though this property is not requested).

Of course there may be other solutions, which possibly save a few nonterminals or rules. For instance one might compact G_2 : remove Z and the rules that have it on the left side, and use the axiomatic rule $S \rightarrow X 0 Y \mid N Y$ instead; this reduces the grammar size. One might also replace N by (S) wherever it occurs in the rules, as well as replace B in the obvious way. Thus one will end up with a highly compacted *BNF* grammar, not ambiguous (axiom S):

$$G'_2 \left\{ \begin{array}{l} S \rightarrow X 0 Y \mid (S) Y \\ X \rightarrow (S) X \mid 0 X \mid 1 X \mid \varepsilon \\ Y \rightarrow (S) Y \mid \varepsilon \end{array} \right.$$

It seems hard to go further and obtain a significantly more compact *BNF* grammar, unless perhaps ambiguity is admitted. This is not explored here.

For completeness, here are two sample syntax trees of grammars G_1 (left) and G_2 (right) for the short strings $0\,1\,((\,1\,0\,)\,1\,)$ and $1\,0\,((\,0\,)\,)$, respectively:



So one sees there is quite many a solution to this exercise, more or less compact.

2. Si vuole modellare grammaticalmente il costrutto **case** dello storico linguaggio di programmazione Pascal, che realizza il condizionale a più vie. Ecco un esempio:

```
case 2 * (x - y) of

    (* branch taken if expression 2 * (x - y) equals 1      *)
    1 : begin          (* branch with two statements *)
        a := b + ( c - 1 ) * d;
        b := c - 1      (* final semicolon is optional *)
    end;

    (* branch taken if expression 2 * (x - y) equals 3 or 4 *)
    3, 4 : a := 2;      (* branch with one statement  *)

    5 : ;               (* a branch may be empty      *)

    -8 : begin          (* blocks may be nested       *)
        ...
        begin          (* block with one statement  *)
            c := -3;    (* final semicolon is optional *)
        end;
        ...
    end;

    (* branch taken in all the other cases                    *)
    otherwise ...      (* final optional branch      *)

end
```

I commenti tra (* e *) non appartengono al linguaggio. Valgono queste specifiche:

- identificatori e costanti sono modellati dai terminali **id** e **num**, rispettivamente
- le espressioni sono numeriche, con variabili e costanti, operatori di addizione '+', sottrazione '-', e moltiplicazione '*', e parentesi tonde '(' e ')'; le precedenze tra operatori sono le solite: l'operatore '*' ha priorità sugli operatori '+' e '-'
- gli assegnamenti ':=' hanno una variabile a sinistra e un'espressione a destra
- il blocco **begin-end** può essere vuoto e in esso il punto-e-virgola ';' è facoltativo prima della parola chiave **end**
- in **case** ci dev'essere almeno un ramo (branch) diverso dalla clausola **otherwise**, che è facoltativa e può comparire solo per ultima
- le etichette di un ramo costituiscono una lista non vuota di costanti numeriche, con separatore virgola ',' e terminatore due-punti ':'
- il costrutto **case** può essere annidato

Se occorre, si immagini di completare ragionevolmente il costrutto **case** descritto prima, motivando eventuali scelte non prescritte dall'esempio o dalle specifiche.

Si scriva una grammatica estesa (*EBNF*), non ambigua, che modella il costrutto **case** di Pascal com'è tratteggiato sopra.

Soluzione

L'esempio è chiaro ed esaustivo. Si può osservare che l'istruzione (statement) è dei tipi seguenti: vuota, assegnamento, blocco **begin-end** (eventualmente annidato), e **case** (ricorsivamente); si veda la regola **STATM**. Ecco la grammatica (assioma **CASE**):

$\langle \text{CASE} \rangle$	$\rightarrow$	case $\langle \text{EXPR} \rangle$ of $\langle \text{BODY} \rangle$ end
$\langle \text{EXPR} \rangle$	$\rightarrow$	$\langle \text{TERM} \rangle$ (('+' '-') $\langle \text{TERM} \rangle$) [*]
$\langle \text{BODY} \rangle$	$\rightarrow$	$\langle \text{BRANCH} \rangle$ (';' $\langle \text{BRANCH} \rangle$) [*] [';' $\langle \text{OTHER} \rangle$]
$\langle \text{TERM} \rangle$	$\rightarrow$	$\langle \text{FACT} \rangle$ ('*' $\langle \text{FACT} \rangle$) [*]
$\langle \text{BRANCH} \rangle$	$\rightarrow$	$\langle \text{LABELS} \rangle$ ':' $\langle \text{STATM} \rangle$
$\langle \text{OTHER} \rangle$	$\rightarrow$	otherwise $\langle \text{STATM} \rangle$
$\langle \text{FACT} \rangle$	$\rightarrow$	id num '(' $\langle \text{EXPR} \rangle$ ')'
$\langle \text{LABELS} \rangle$	$\rightarrow$	num (',' num) [*]
$\langle \text{STATM} \rangle$	$\rightarrow$	ε $\langle \text{ASGN} \rangle$ $\langle \text{BLOCK} \rangle$ $\langle \text{CASE} \rangle$
$\langle \text{ASGN} \rangle$	$\rightarrow$	id ':' $\langle \text{EXPR} \rangle$
$\langle \text{BLOCK} \rangle$	$\rightarrow$	begin $\langle \text{STATM} \rangle$ (';' $\langle \text{STATM} \rangle$) [*] end

Le parentesi quadre indicano opzionalità. Come in tanti altri linguaggi di programmazione, il blocco **begin-end** (o un costrutto simile delimitato da altri simboli) contiene una o più istruzioni; esso è facoltativo se c'è una sola istruzione. Tuttavia, dato che l'istruzione vuota è ammessa, il blocco può anche essere vuoto.

La regola **BLOCK** non mette il separatore punto-e-virgola prima della parola chiave **end** che chiude il costrutto **begin-end**, ma al termine delle istruzioni contenute in **begin-end** il separatore risulterà facoltativo poiché l'istruzione vuota è ammessa.

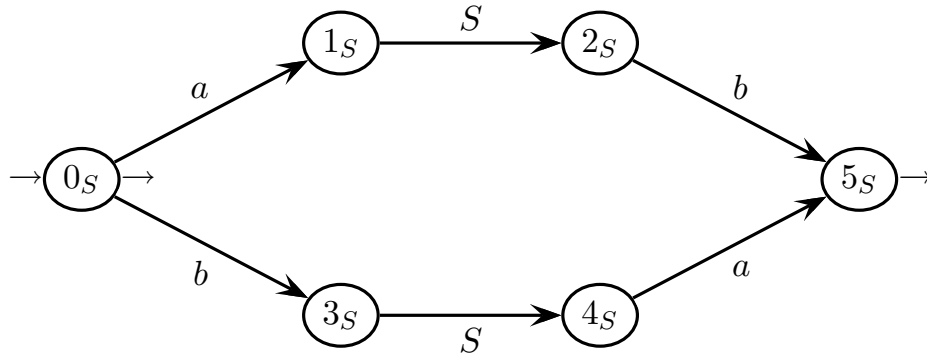
L'esempio suggerisce che prima della parola chiave **end** che chiude il costrutto **case**, non ci sia separatore punto-e-virgola. Coerentemente con tale osservazione, per la grammatica proposta il separatore è impossibile prima dello **end** finale di **case**: la regola **CASE** non lo inserisce e un ramo (branch) vuoto non è ammesso. Il linguaggio Pascal è strutturato proprio così, tuttavia si tratta di un dettaglio.

La grammatica è in forma estesa (*EBNF*), e non è ambigua poiché è costituita da componenti notoriamente non ambigui. Sono certamente possibili varianti, puramente sintattiche o magari con piccole differenze interpretative dell'esempio.

3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica G seguente (assioma S), rappresentata mediante la sua rete di macchine, che qui si riduce a una sola macchina:

$$G: S \rightarrow a S b \mid b S a \mid \varepsilon$$



Si risponda alle domande seguenti, usando la rappresentazione come macchina:

- (a) Si mostri che la grammatica G non è $ELR(1)$ costruendo la più piccola porzione possibile dell'automa pilota.
- (b) Si applichi la procedura di costruzione diretta e semplificata del grafo di controllo $PCFG$ della grammatica G , e si mostri su di esso che la grammatica non è $ELL(1)$. Si spieghi in dettaglio il calcolo degli insiemi guida significativi.
- (c) Applicando il metodo di Earley alla grammatica G , si analizzi la stringa $abab$. Con riferimento al contenuto del vettore E si mostri che la stringa è accettata, come pure la stringa prefisso ab , mentre la stringa prefisso aba è rifiutata. Si usi la tabella predisposta di seguito (il numero di righe non è significativo).
- (d) (facoltativa) Si dica se il linguaggio $L(G)$ generato dalla grammatica G è deterministico, argomentando la risposta.

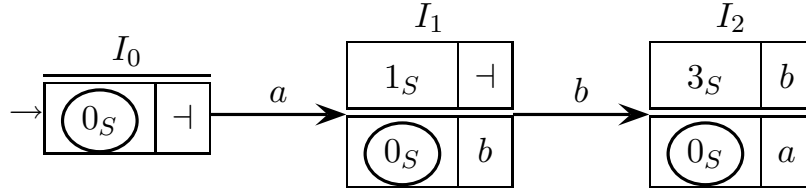
PER QUANTO SEGUE VEDI ANCHE AVVISO SUL FRONTESPIZIO DEL TESTO

Chi preferisse rispondere con la metodologia classica di analisi LR e LL , usi la grammatica G , che è già in forma non estesa, invece della rete di macchine.

[illegible]

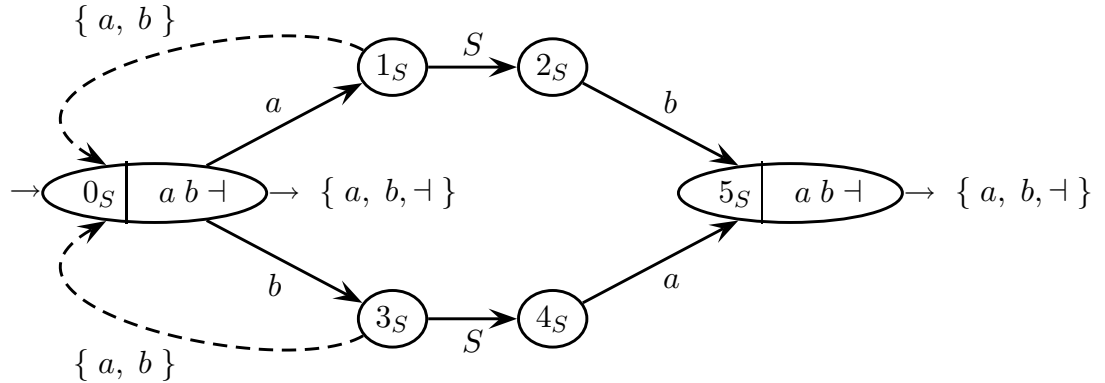
Soluzione

- (a) Ecco il grafo pilota della grammatica G , sviluppato quanto basta per decidere il determinismo:



Già con tre soli m-stati si vede che nello m-stato I_1 c'è un conflitto di tipo shift-reduce sulla lettera b . Pertanto la grammatica G non è di tipo $ELR(1)$. Il grafo pilota completo è assai più grande (il lettore lo può costruire per esercizio).

- (b) Ecco il grafo $PCFG$ della grammatica G , costruito direttamente:



L'insieme di prospezione negli stati finali 0_S e 5_S è $\{a, b, \vdash\}$. Infatti il nonterminale S è seguito dai terminali a e b uscendo dagli stati 4_S e 2_S , rispettivamente; inoltre, essendo S assioma, c'è anche il terminatore $\vdash$. L'insieme di prospezione coincide con l'insieme guida sulla freccia di uscita degli stati finali.

Entrambi gli insiemi guida sugli archi di chiamata (call arc) a S uscendo dagli stati 1_S e 3_S , sono $\{a, b\}$. Infatti il nonterminale S inizia con i due terminali a e b , e tanto basta per includerli nell'insieme guida. Esso è nullabile, ma è necessariamente seguito da a e b uscendo da 4_S e 2_S , rispettivamente, e quei due terminali sono già nell'insieme guida. Non ci sono altri terminali da aggiungere (e certamente il terminatore $\vdash$ non va aggiunto).

Si vede subito che nello stato 0_S l'insieme di prospezione o equivalentemente l'insieme guida sulla freccia di uscita, ha elementi in comune con i due archi di spostamento a e b verso gli stati 1_S e 3_S , rispettivamente. Dunque in 0_S c'è conflitto (in termini ELR sarebbe di tipo spostamento-riduzione), pertanto il grafo $PCFG$ non è deterministico e la grammatica G non è di tipo $ELL(1)$.

- (c) Ecco il vettore di Earley completo, con cinque elementi $E(i)$ ($0 \leq i \leq 4$), per la stringa $a b a b$ da analizzare:

$E(0)$	a	$E(1)$	b	$E(2)$	a	$E(3)$	b	$E(4)$
$\textcircled{0_S} 0$		$1_S 0$		$3_S 1$		$1_S 2$		$3_S 3$
		$\textcircled{0_S} 1$		$\textcircled{5_S} 0$		$\textcircled{5_S} 1$		$\textcircled{5_S} 0$
		$2_S 0$		$\textcircled{0_S} 2$		$\textcircled{0_S} 3$		$\textcircled{5_S} 2$
				$4_S 1$		$2_S 0$		$\textcircled{0_S} 4$
						$2_S 2$		$4_S 1$
								$4_S 3$

Poiché nell'elemento $E(4)$ del vettore c'è la candidata finale $\langle \textcircled{5_S} 0 \rangle$ che si riferisce all'inizio del vettore, la stringa $a b a b$ è accettata. Similmente, nell'elemento $E(2)$ c'è la candidata finale $\langle \textcircled{5_S} 0 \rangle$ che si riferisce all'inizio del vettore, dunque anche la stringa $a b$ è accettata. Invece, nell'elemento $E(3)$ c'è la candidata finale $\langle \textcircled{5_S} 1 \rangle$ che però non si riferisce all'inizio del vettore, e c'è anche la candidata finale $\langle \textcircled{0_S} 3 \rangle$ che però si riferisce all'elemento stesso (dunque essa corrisponde a riconoscere in loco la stringa vuota ε); in $E(3)$ non ci sono altre candidate finali e in particolare non ce n'è nessuna che si riferisca all'inizio del vettore; pertanto la stringa $a b a$ è rifiutata.

Tramite frecce è rappresentata anche la ricostruzione all'indietro dell'albero sintattico della stringa $a b a b$ da analizzare: in **rosso** gli spostamenti terminali; in **rosa** gli spostamenti nonterminali; e in **blu** la corrispondenza tra spostamento nonterminale e riduzione che lo ha abilitato.

Per completezza si noti che le varie altre candidate finali con stato $\textcircled{0_S}$ corrispondono tutte, tranne la prima, a elementi non iniziali per il vettore; pertanto esse non riconoscono alcun prefisso della stringa $a b a b$ da analizzare. La prima riconosce la stringa vuota ε che ovviamente è sempre prefisso, però banale.

- (d) Si può osservare che il linguaggio $L(G)$ generato dalla grammatica G , somiglia molto al linguaggio delle palindromi senza centro: ogni stringa di $L(G)$ ha la forma uv , dove si ha $v = h(u)^R$ e la traslitterazione h scambia le lettere a e b . La traslitterazione h non fa problema al determinismo poiché a lettere diverse fa corrispondere lettere diverse. Ragionevolmente pertanto, l'analisi di $L(G)$ deve procedere sulla falsariga di quella delle palindromi senza centro, ossia stringhe come $u u^R$, che notoriamente costituiscono un linguaggio nondeterministico. Se ne conclude che verosimilmente il linguaggio $L(G)$ non è deterministico.

4 Traduzione e analisi semantica 20%

1. Nel linguaggio sorgente L_s ogni frase ha la struttura $(a^+ b^+)^+$. Pertanto la generica frase di L_s si presenta come una serie di blocchi, così:

$$a^{m_1} b^{n_1} \dots \overbrace{a^{m_i} b^{n_i}}^{\text{blocco}} \dots a^{m_k} b^{n_k} \quad \text{con } k \geq 1$$

Si consideri la traduzione τ tale che ogni *blocco* è trasformato così:

$$a^{m_i} b^{n_i} \xrightarrow{\tau} \begin{cases} c^{m_i+n_i} & \text{se } m_i \geq n_i \\ d^{n_i-m_i} & \text{se } m_i < n_i \end{cases}$$

Esempio:

$$a^2 b^2 a^2 b^3 a^3 b \xrightarrow{\tau} c^4 d c^4$$

Si risponda alle domande seguenti:

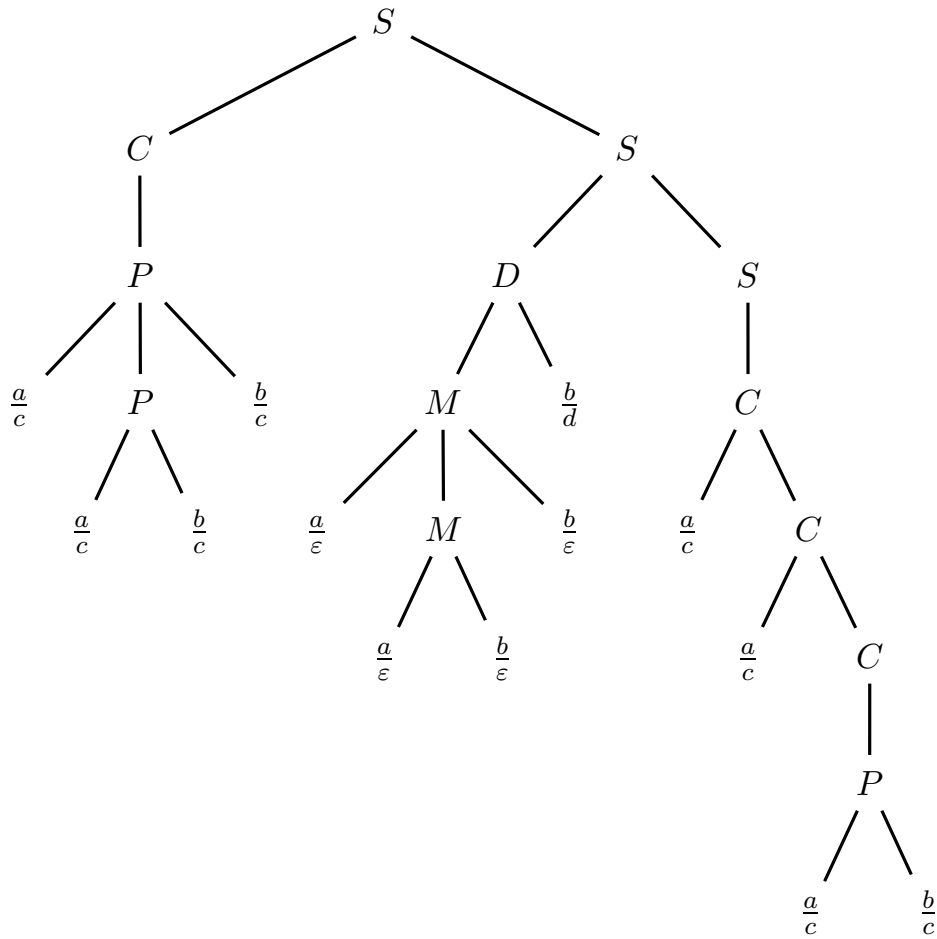
- (a) Si scriva uno schema puramente sintattico (G_s, G_d) o equivalentemente una grammatica di traduzione G_τ , per definire la traduzione τ ; si disegnino gli alberi sintattici sorgente e destinazione (o unificandoli) per la stringa $a^2 b^2 a^2 b^3 a^3 b$.
- (b) (facoltativa) Si dica se la traduzione τ è calcolabile mediante un automa traduttore a pila deterministico A_τ , argomentando la risposta.

Soluzione

(a) The translation grammar G_τ below, defines the requested translation (axiom S):

$$G_\tau \left\{ \begin{array}{l} S \rightarrow C S \mid D S \mid C \mid D \\ C \rightarrow \frac{a}{c} C \mid P \\ D \rightarrow D \frac{b}{d} \mid M \frac{b}{d} \\ P \rightarrow \frac{a}{c} P \frac{b}{c} \mid \frac{a}{c} \frac{b}{c} \\ M \rightarrow \frac{a}{\varepsilon} M \frac{b}{\varepsilon} \mid \frac{a}{\varepsilon} \frac{b}{\varepsilon} \end{array} \right.$$

Here we draw a unified source and destination syntax tree for the sample translation $a^2 b^2 a^2 b^3 a^3 b \xrightarrow{\tau} c^4 d c^4$ (one might split it if preferable).



The autoinclusive rule P generates equal numbers of source letters a and b in a block, and translates them into as many destination letters c ; while the right-linear rule C generates the possible additional a 's in the block and also

translates them into as many c 's; so in this case (which is $m \geq n$) the length of the block translation amounts to the sum of the numbers of its a 's and b 's. The autoinclusive rule M generates as P does in the source, but it has an empty destination; while the right-linear rule D generates the additional b 's (at least one) in the block and translates them into as many destination letters d ; so in this case (which is $m < n$) the length of the block translation amounts to the difference of the numbers of its b 's and a 's. The axiomatic rule, which is also right-linear, freely generates a sequence of block translations, where each block is treated according to either case. This is a sufficient justification to prove the correctness of the translation grammar G_τ . The sample syntax tree above helps us visualize the grammar operation.

- (b) To answer this question, one may consider a syntax analyser driven by the translation grammar G_τ or directly reason on a generic pushdown transducer A_τ . The source grammar G_s that underlies grammar G_τ , is below (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow C S \mid D S \mid C \mid D \\ C \rightarrow a C \mid P \\ D \rightarrow D b \mid M b \\ P \rightarrow a P b \mid a b \\ M \rightarrow a M b \mid a b \end{array} \right.$$

With an easy reasoning, grammar G_s does not have the property *ELR* (or *LR*). In fact the languages generated by the nonterminals C and D contain the strings $a^n b^n$ and $a^n b^{n+1}$ ($n \geq 1$), respectively, and the source language $L(G_s)$ includes the union of the languages of C and D . As far as the pilot of the syntax analyser shifts on the n initial letters a , it will carry on in parallel both the derivations that start by C and D and go on by P and M , respectively. But when the analyser encounters the first letter b , it will be unable to decide between reductions $a b \rightsquigarrow P$ and $a b \rightsquigarrow M$; essentially, it exhibits a reduce-reduce conflict. Therefore a syntax transducer driven by G_τ , is not deterministic.

On the other hand, we argue a generic pushdown transducer A_τ cannot be deterministic. We know that each letter a encountered, has to be either translated into c or ignored, depending on the inequality $m \geq n$ being *true* or *false* for the block under processing. Two design decisions are possible: either to immediately emit the c 's or to push the a 's onto the stack. The former strategy outputs a wrong translation in the case *false*, and so we have to opt for the latter one.

When machine A_τ reads the first b , the stack contains m symbols a . Clearly emitting the c 's would be too early since the truth status of inequality $m \geq n$ is not known yet. Moreover machine A_τ cannot output the translation for the incoming b 's as it does not know whether it should be c or d . Machine A_τ is only left the possibility of pushing also the b 's onto the stack. But with so doing, machine A_τ is prevented of verifying the inequality because the only way would be to pop one a for each incoming b and test if the stack is empty or not before reading the last b . We reasonably conclude that a pushdown transducer A_τ able to compute this translation deterministically, does not exist.

2. Si vuole tradurre un'espressione da forma infissa a forma postfissa, servendosi di una grammatica con attributi invece che di uno schema sintattico di traduzione. Ecco la sintassi dell'espressione da tradurre, aritmetica associativa a destra (assioma S):

$$\left\{ \begin{array}{ll} S \rightarrow E & \\ E \rightarrow T + E & T \rightarrow (E) \\ E \rightarrow T & T \rightarrow a \end{array} \right.$$

Il modello di espressione è molto semplice e comprende solo l'operatore di addizione e le parentesi. Il terminale a rappresenta genericamente variabili e costanti.

In particolare sono previste due traduzioni diverse ρ e σ , così:

- La traduzione ρ calcola l'usuale forma postfissa (associando a destra) e naturalmente *elimina* le parentesi. Esempio:

la stringa sorgente $a +_1 a +_2 (a +_3 a) +_4 a$

si traduce in $\rho = a a a a +_3 a +_4 +_2 +_1$

- La traduzione σ calcola l'usuale forma postfissa (associando a destra), ma solo tra un numero *dispari* di parentesi annidate, mentre altrove (ossia tra un numero pari di parentesi) lascia la forma infissa, e naturalmente *conserva* le parentesi. Le parti di espressione esterne a parentesi sono considerate a livello pari di annidamento. Esempio:

la stringa sorgente $a +_1 (a +_2 (a +_3 a) +_4 a)$

si traduce in $\sigma = a +_1 (a (a +_3 a) a +_4 +_2)$

Nei due esempi dati sopra, la numerazione dell'operatore '+' è solo un aiuto visivo per capire come funziona la traduzione, ma non appartiene alla grammatica.

In entrambi i casi l'espressione tradotta va rappresentata tramite un attributo τ di tipo stringa, emesso a livello della radice dell'albero sintattico. Si possono aggiungere altri attributi, se lo si ritiene necessario od opportuno. Le tabelle da compilare con gli attributi e le regole semantiche, sono già predisposte di seguito.

Si risponda alle domande seguenti:

- Si scriva una grammatica con attributi G_ρ che calcola la stringa traduzione ρ .
- Si scriva una grammatica con attributi G_σ che calcola la stringa traduzione σ , e si precisi se la grammatica G_σ è di tipo one-sweep (a una passata).
- (facoltativa) Si decori l'albero sintattico della stringa di esempio per la traduzione σ , con gli attributi, i loro valori e le dipendenze funzionali tra attributi. L'albero da decorare è già dato nel seguito.

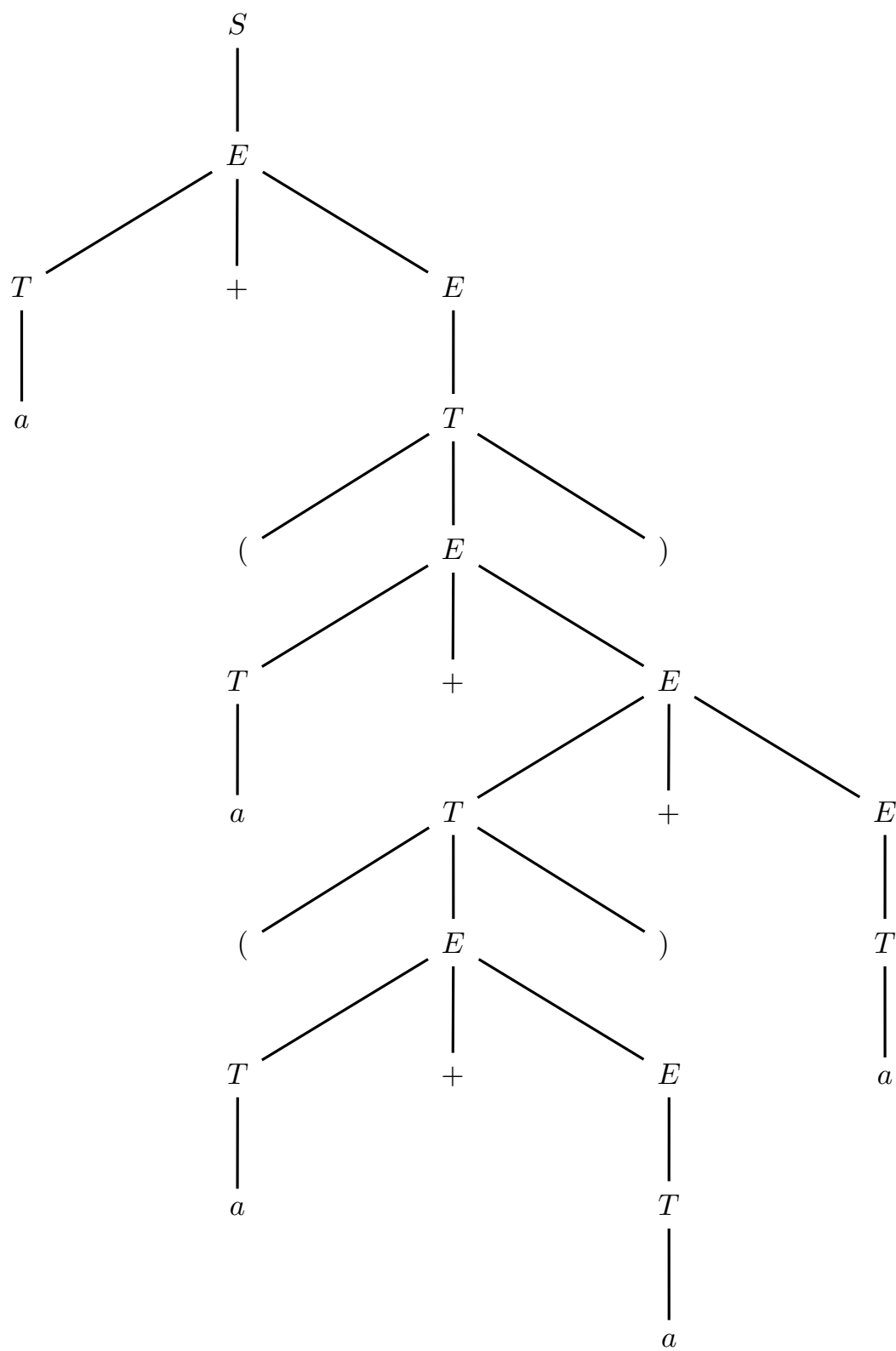
attributi da usare per le grammatiche - domande (a) e (b)

tipo	nome	(non)terminali	dominio	significato
sin	τ	S, E, T	stringa	traduzione (secondo ρ o σ)

sintassi	calcolo attributi per la traduzione ρ - domanda (a)
$S_0 \rightarrow E_1$	
$E_0 \rightarrow T_1 + E_2$	
$E_0 \rightarrow T_1$	
$T_0 \rightarrow (E_1)$	
$T_0 \rightarrow a$	

sintassi	calcolo attributi per la traduzione σ - domanda (b)
$S_0 \rightarrow E_1$	
$E_0 \rightarrow T_1 + E_2$	
$E_0 \rightarrow T_1$	
$T_0 \rightarrow (E_1)$	
$T_0 \rightarrow a$	

albero sintattico di $a + (a + (a + a) + a)$ da decorare



Soluzione

- (a) La grammatica con attributi G_ρ è molto semplice e puramente sintetizzata: basta l'attributo sinistro τ (già assegnato nelle specifiche) calcolato secondo la regola postfissa posponendo l'operatore di addizione. Ecco la grammatica:

sintassi	calcolo attributi per la traduzione ρ - domanda (a)
$S_0 \rightarrow E_1$	$\tau_0 = \tau_1$
$E_0 \rightarrow T_1 + E_2$	$\tau_0 = \tau_1 \cdot \tau_2 \cdot '+'$ - regola postfissa
$E_0 \rightarrow T_1$	$\tau_0 = \tau_1$
$T_0 \rightarrow (E_1)$	$\tau_0 = \tau_1$ - elimina parentesi
$T_0 \rightarrow a$	$\tau_0 = 'a'$

L'operatore $'.'$ concatena le varie istanze dell'attributo stringa τ e le stringhe costanti che derivano dai terminali della grammatica. La funzione semantica associata alla regola sintattica di addizione calcola la forma postfissa e quella associata alla regola con parentesi, le elimina; il resto è ovvio.

- (b) La grammatica con attributi G_σ è piuttosto semplice: la base è fornita dalla grammatica G_ρ costruita prima, cui basta aggiungere un attributo ereditato (destro) booleano λ che commuta tra livello di annidamento pari (*false*) e dispari (*true*) attraversando le parentesi, e che dirige il calcolo dell'attributo τ selezionando la regola infissa o postfissa. Ecco gli attributi e la grammatica:

attributi da usare per le grammatiche

tipo	nome	(non)terminali	dominio	significato
sin	τ	S, E, T	stringa	traduzione (secondo ρ o σ)
des	λ	S, E, T	booleano	parità del livello di annidamento tra le parentesi

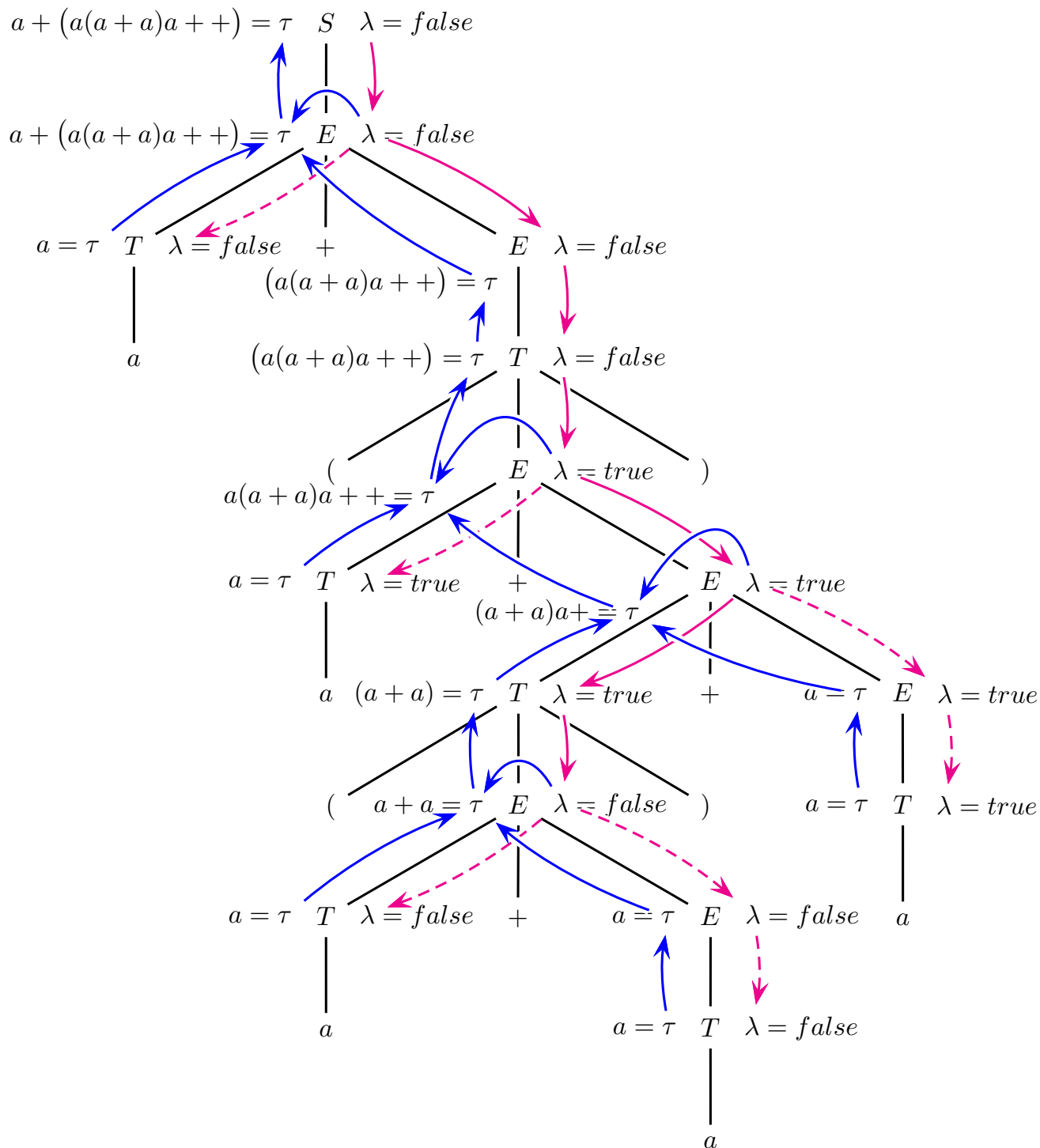
sintassi	calcolo attributi per la traduzione σ - domanda (b)
$S_0 \rightarrow E_1$	$\lambda_1 = false$ - inizializzazione $\tau_0 = \tau_1$
$E_0 \rightarrow T_1 + E_2$	$\lambda_1 = \lambda_0$ $\lambda_2 = \lambda_0$ if $\lambda_0 = true$ then $\tau_0 = \tau_1 \cdot \tau_2 \cdot '+'$ - regola postfissa else $\tau_0 = \tau_1 \cdot '+' \cdot \tau_2$ - regola infissa endif
$E_0 \rightarrow T_1$	$\lambda_1 = \lambda_0$ $\tau_0 = \tau_1$
$T_0 \rightarrow (E_1)$	$\lambda_1 = \mathbf{not} \lambda_0$ - commutazione $\tau_0 = '(' \cdot \tau_1 \cdot ')'$ - conserva parentesi
$T_0 \rightarrow a$	$\tau_0 = 'a'$

I commenti mettono in evidenza somiglianze e differenze tra la grammatica G_σ e la più semplice G_ρ . La funzione semantica associata alla regola sintattica di addizione è formulata come condizionale e calcola la forma postfissa o infissa in alternativa. Le funzioni semantiche associate alla regola sintattica con parentesi, complementano la parità di livello di annidamento tra parentesi e conservano le parentesi stesse. La parità di livello è inizializzata a pari (*false*) nella regola sintattica assiomatica. Il resto è pure ovvio.

In G_σ è evidente che prima si scende calcolando l'attributo ereditato λ , e poi si sale calcolando l'attributo sintetizzato τ in modo postfisso o infisso secondo il valore di λ . Pertanto la grammatica G_σ è di tipo one-sweep (a una passata).

- (c) L'albero sintattico è semplice da decorare, una volta progettata la grammatica con attributi G_σ . Eccolo, con il calcolo degli attributi ereditati e sintetizzati in **rosa** e **blu**, rispettivamente:

albero sintattico decorato della stringa $a + (a + (a + a) + a)$ da tradurre



Si vede bene il calcolo della traduzione τ , alternativamente postfisso e infisso secondo l'attributo λ . Alcune propagazioni di λ risultano inutili (freccie **rosa** tratteggiate) poiché non ci sono più operatori contenuti, ma naturalmente l'attributo λ è definito in tutti i nodi nonterminali e formalmente va calcolato ovunque.